

Software Technology 1 (4483)
Assessment 3: Part A - Implementation
and Summary
U3306631, U3308882, U3309505
Group 8 – Thursday 2:30

2. Brief Project Goal	2
3. System Design Overview.....	2
4. Class and Module Overview	2
5. Python Packages used and why	2
6. Key Features Implemented.....	2
7. Screenshots or Example Outputs.....	2
8. Testing Summary	2
9. Acknowledgement of reused or adapted code.....	2
10. Short Summary of how the work was distributed across the group	2

2. Brief Project Goal

This project is a Python based modular application designed to analyse a macroinvertebrate image dataset in a stream. It loads raw image data into a structured table, conducts EDA to show the distribution of classes, size variation, and data characteristics and trains a baseline image classifier using the Random Forest algorithm. The trained model is deployed in two interfaces, both of which are accessible by a user: a Tkinter-based desktop GUI that show the user a dropdown menu that lists all images in the database and allows the user to select any image he wants to get a prediction for the class of macroinvertebrate found and its confidence score, and a menu-driven console application that also displays a dropdown menu listing all the images in the database to the user and provides him/her the ability of selecting any image to obtain a prediction on the class of macroinvertebrate present in that image and the confidence score.

3. System Design Overview

It's an object-oriented Python application with distinct layers of responsibility, is designed as a modular, object-oriented Python application, divided into distinct layers of responsibility. There are no hard coded values throughout the code base; all shared paths and constants are centralised in config.py. There are three sub-packages in the src/, models/ containing data classes, services/ with business logic, and entry-point scripts (main.py, console_app.py, app.py) that remain lightweight while passing all work to a centralised and shared WorkflowService.

Stage 1 is on Data Set Indexing and Exploratory Data Analysis. In Stage 3 there are two options for deployment: a Tkinter GUI (app.py) and a menu-driven console application (console_app.py). The same WorkflowService is used in both entry points, no duplicated logic is implemented in both.

4. Class and Module Overview

Class / Module	Responsibility
config.py	Centralises all paths, image settings, and model so values are defined once and imported everywhere
ImageRecord (models/records.py)	A @dataclass that holds the meta data for one indexed image: file path, class label, width, height, and channel count
DatasetIndexer (services/dataset_indexer.py)	Recursively scans the raw data from folder, reads each image with OpenCV, and returns a Pandas DataFrame with one row per valid image
EDAService (services/eda_service.py)	Generates and saves seven visualisation outputs , class distribution bar chart, width/height histograms, pie chart, channel distribution, aspect ratio histogram, sample image grid, and a width-vs-height scatter plot, plus a summary statistics dictionary
ImagePreprocessor (services/image_preprocessor.py)	Converts each raw image into a model-ready feature vector by reading it as grayscale, resising to 128×128, normalising pixel values to [0, 1], and flattening to a 1-D array
ClassifierService (services/classifier_service.py)	Builds feature matrices from the indexed DataFrame, trains a RandomForestClassifier with a stratified 80/20 split, returns accuracy and a full classification report, and saves/loads the model with joblib
WorkflowService (services/workflow_service.py)	Coordinates all shared actions (summary, EDA generation, training) and is the single point of contact for all three entry points
ConsoleApp (console_app.py)	Provides a menu-driven terminal interface with five options: Select Dataset, Generate stage 1 EDA, Choose image, show image detail, Show EDA output Location, Exit
MacroApp (app.py)	Tkinter desktop GUI allowing the user to browse for an image, preview it, and view the predicted class and confidence

5. Python Packages used and why

Package	How it was used in this project
pathlib	All file and folder paths are constructed as Path objects throughout the project, making cross-platform path handling clean and readable

pandas	The DatasetIndexer returns a DataFrame of image records, this table is passed directly into the EDA and training stages, making filtering and statistics straightforward
numpy	The ImagePreprocessor returns np.ndarray feature vectors, ClassifierService stacks these into a feature matrix with np.array() for use by scikit-learn
opencv-python	Used in DatasetIndexer to read images and extract their dimensions, and in ImagePreprocessor to load images as grayscale and resize them to 128×128
matplotlib & seaborn	Used in EDAService to produce all seven charts, including count plots, histograms with KDE overlays, scatter plots, and pie charts
scikit-learn	Provides RandomForestClassifier for training, train_test_split with stratification for a fair evaluation split, and accuracy_score, classification_report, and confusion_matrix for evaluation
joblib	Saves the trained RandomForestClassifier to disk as macro_classifier.joblib and reloads it for prediction without retraining
Pillow	Used in the Tkinter GUI (app.py) to open images and create ImageTk.PhotoImage objects for display in the interface

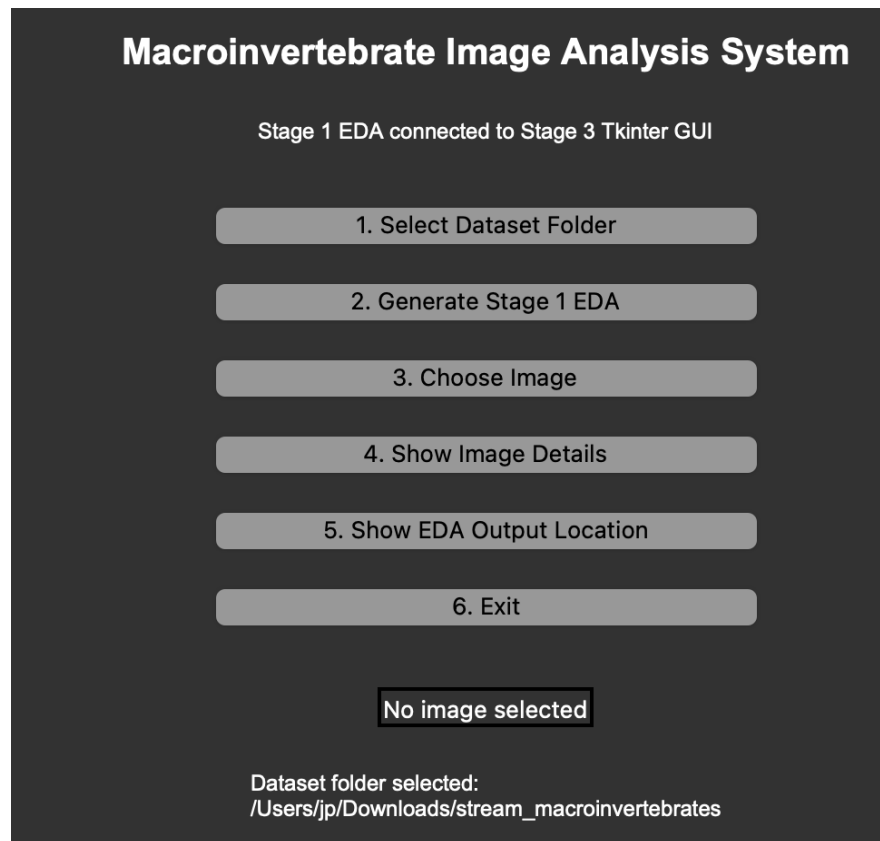
6. Key Features Implemented

The key features implemented in our project include dataset indexing, exploratory data analysis, image visualisation, and a Tkinter desktop GUI menu application. The system scans the macroinvertebrate dataset and stores image information such as file paths, labels, image widths and heights into a structured pandas DataFrame. The application generates several exploratory data analysis and visualisation (EDA) outputs including a dataset summary, class distribution chart, image size distribution chart, and a sample image grid for data inspection.

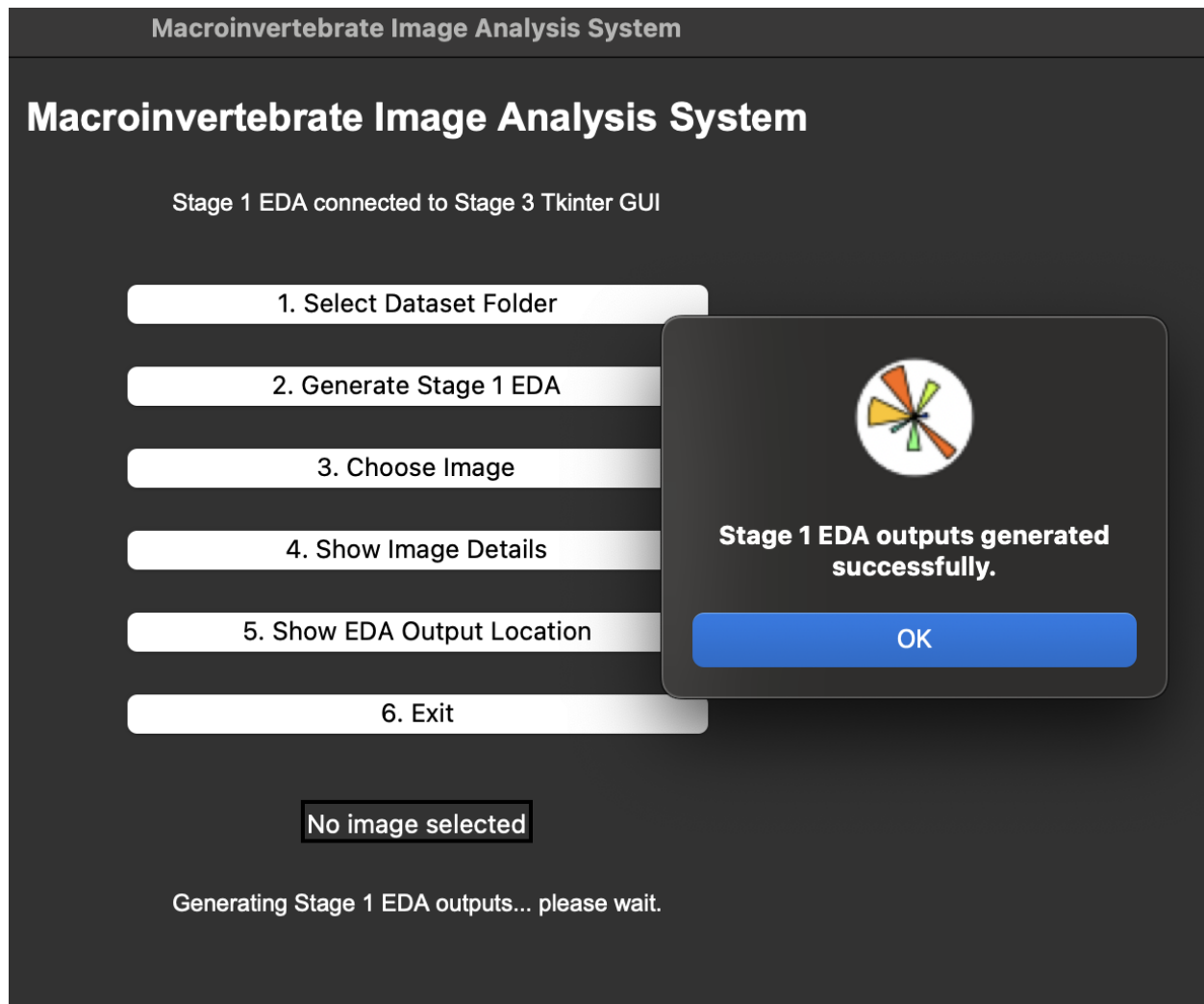
A Tkinter GUI menu was developed to allow users to interact with the system through a simple desktop interface. The GUI supports dataset folder selection, Stage 1 EDA generation, image selection, image preview, and image detail display. The project was designed using object-oriented programming principles with separate classes and modules to improve readability and maintainability. The stage 1 EDA module was integrated directly into the GUI to create one completed and connected application.

7. Screenshots or Example Outputs

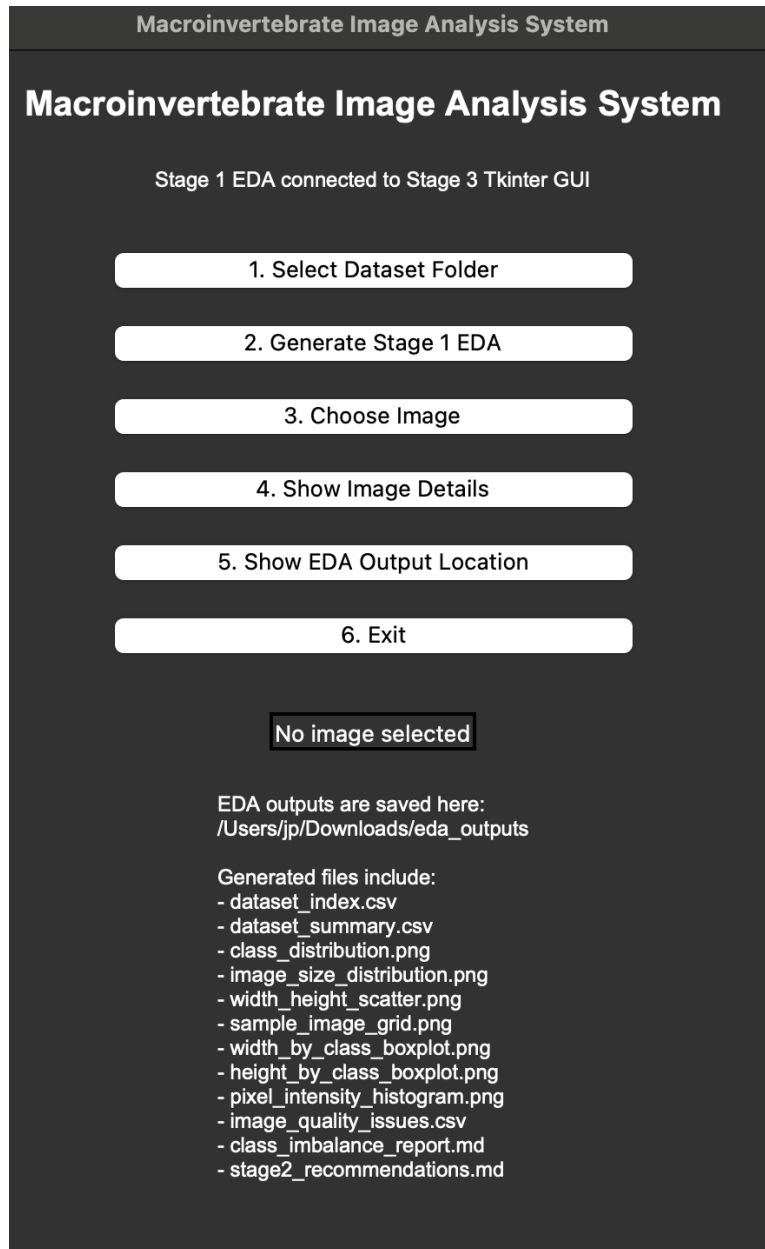
Opening the GUI menu and selecting a dataset folder:

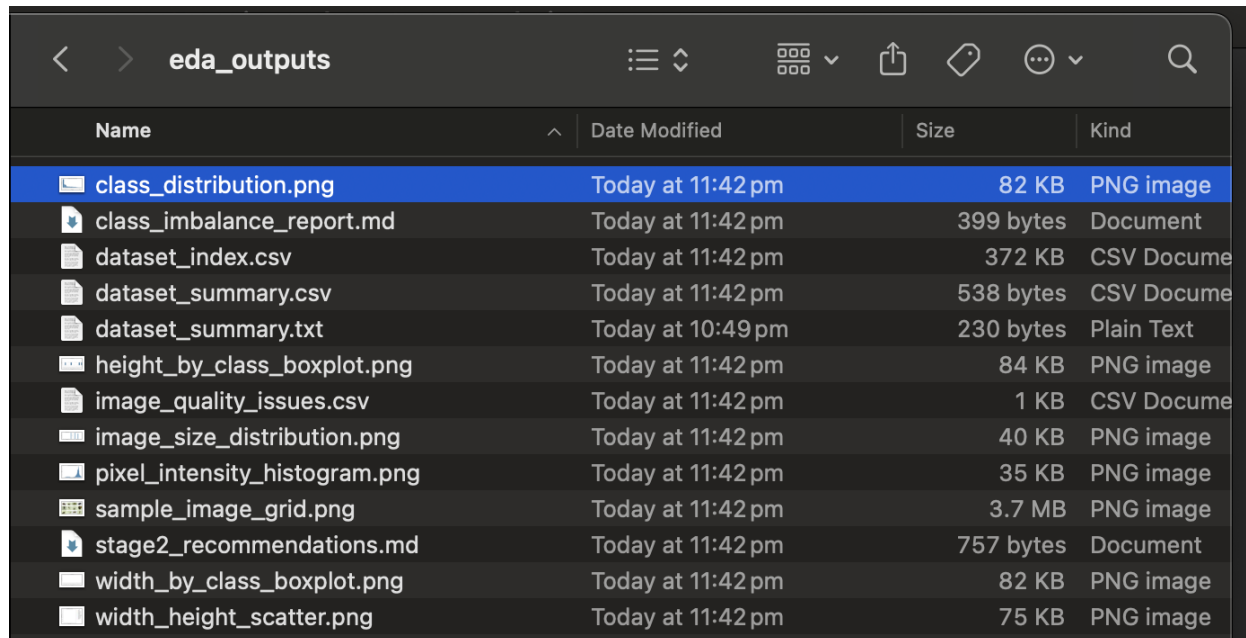


Generating EDA outputs

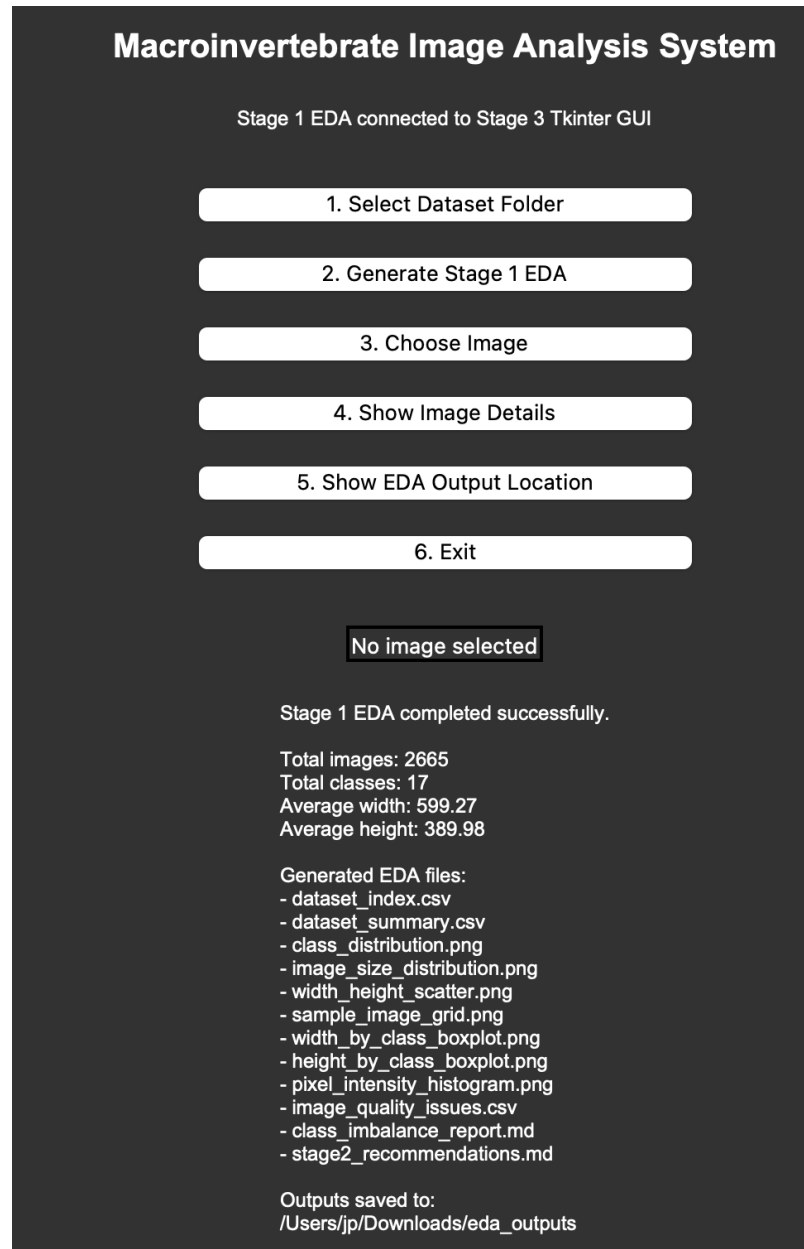


Successful test of EDA outputs

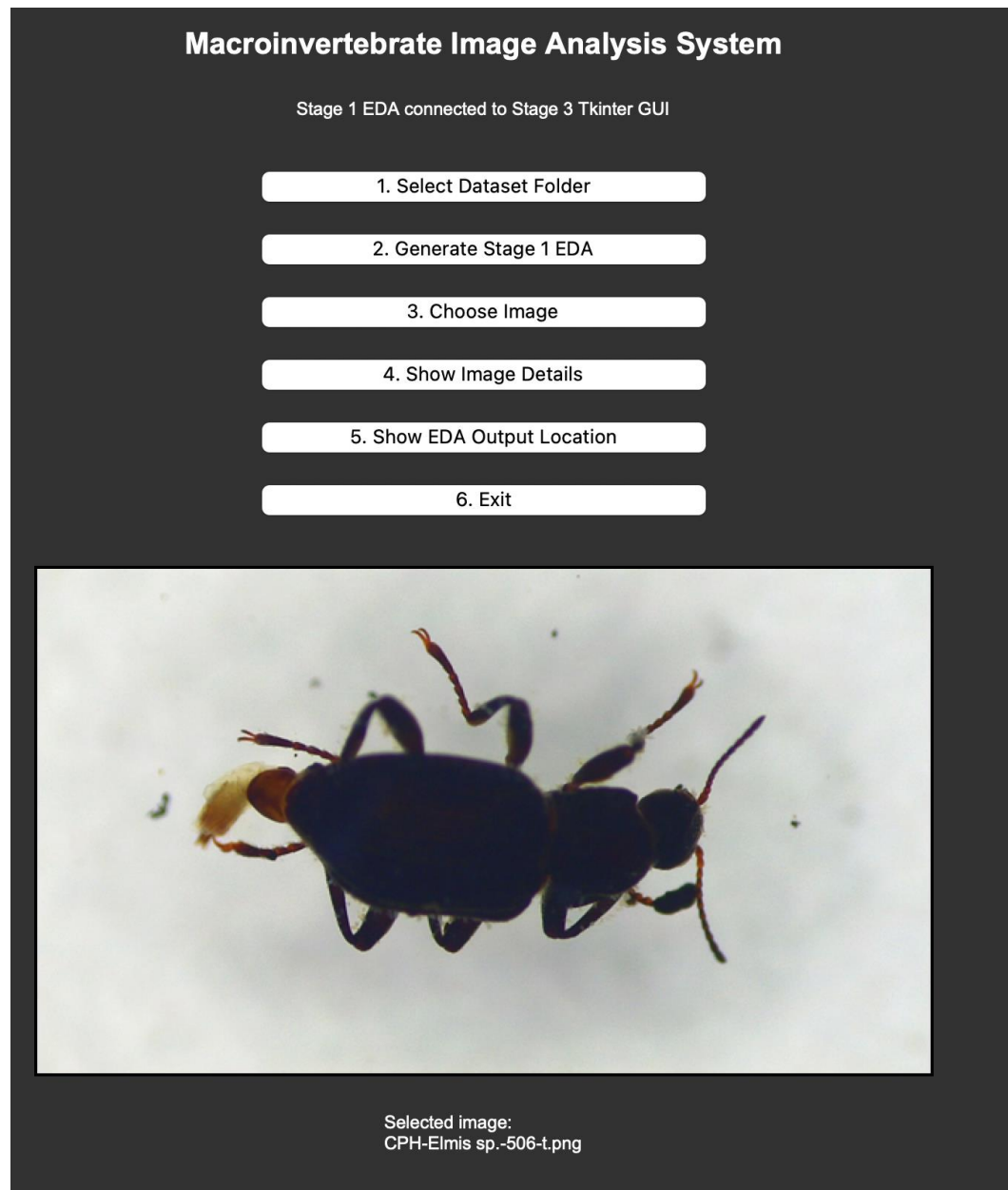




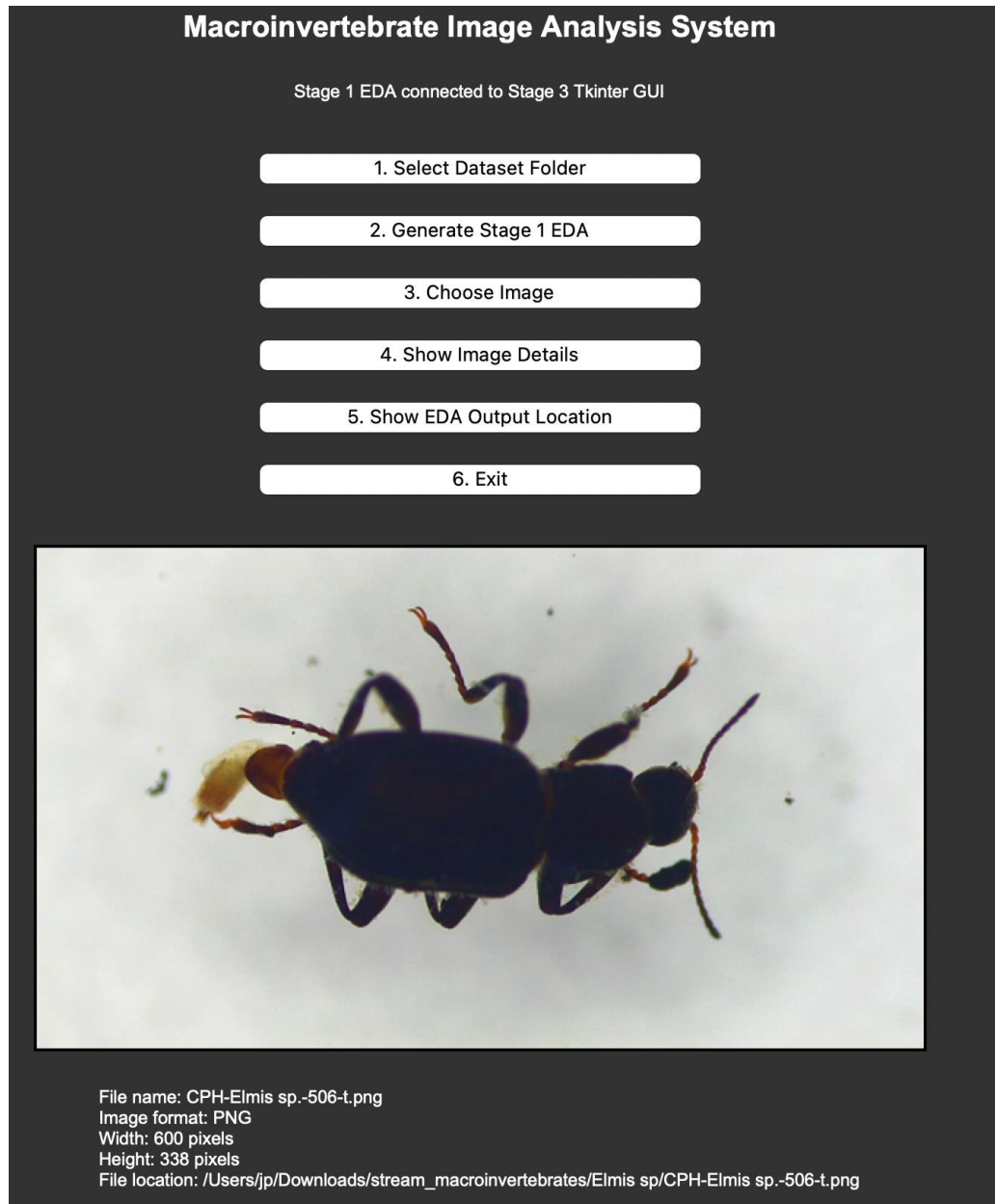
Name	Date Modified	Size	Kind
 class_distribution.png	Today at 11:42 pm	82 KB	PNG image
 class_imbalance_report.md	Today at 11:42 pm	399 bytes	Document
 dataset_index.csv	Today at 11:42 pm	372 KB	CSV Docume
 dataset_summary.csv	Today at 11:42 pm	538 bytes	CSV Docume
 dataset_summary.txt	Today at 10:49 pm	230 bytes	Plain Text
 height_by_class_boxplot.png	Today at 11:42 pm	84 KB	PNG image
 image_quality_issues.csv	Today at 11:42 pm	1 KB	CSV Docume
 image_size_distribution.png	Today at 11:42 pm	40 KB	PNG image
 pixel_intensity_histogram.png	Today at 11:42 pm	35 KB	PNG image
 sample_image_grid.png	Today at 11:42 pm	3.7 MB	PNG image
 stage2_recommendations.md	Today at 11:42 pm	757 bytes	Document
 width_by_class_boxplot.png	Today at 11:42 pm	82 KB	PNG image
 width_height_scatter.png	Today at 11:42 pm	75 KB	PNG image



Selecting an image:



Showing image details of selected image



8. Testing Summary

The application was tested throughout development to ensure the Stage 1 EDA and Stage 3 GUI features functioned correctly. Testing included verifying the dataset folder selection, image loading, image preview functionality, image detail display, and EDA output generation. The dataset indexing process was tested to confirm that image metadata such as labels, image widths, and heights were correctly stored in the pandas

DataFrame. The EDA visualisations including the class distribution chart, image size distribution chart, and sample image grid were also tested to ensure they generated correctly and saved to the output folder. During development, some issues were encountered with incorrect dataset paths and GUI responsiveness while generating EDA outputs. These issues were resolved by correcting dataset folder selection and improving integration between the GUI and Stage 1 EDA module.

9. Acknowledgement of reused or adapted code

Unit tutorial examples and weekly lab materials were referenced during development to help guide the structure and implementation of the project. The final system was modified and integrated to suit the requirements of the Macroinvertebrate Image Analysis System.

10. Short Summary of how the work was distributed across the group

Work was divided across the group to improve organisation and development efficiency. One member focused mainly on the Stage 1 EDA implementation including dataset indexing, dataframe creation, and visualisation generation. Another member focused on the Tkinter GUI development including image selection, image preview, and interface integration. Group members also collaborated on testing, debugging, system integration, and preparation of the final presentation and implementation summary. Regular communication was used to ensure that the final application remained consistent and properly integrated.